Experiment: 7

Aim: Implement Queues, Circular Queues.

1. Queue ADT

Definition:

A queue is a linear data structure that follows FIFO (First In First Out) principle: the first element inserted is the first to be removed.

Basic Operations of Queue:

1. enqueue(element): Insert element at the rear.
2. dequeue(): Remove element from the front.
3. peek/front(): Return the front element without removing it.
4. isEmpty(): Check if the queue is empty.

Applications:

- Print job management
- CPU scheduling
- BFS traversal in graphs

Algorithm for Queue Operations

Enqueue:

1. Check if the queue is full (for fixed-size queue).
2. If not full, increment `rear` and insert element.

Dequeue:

1. Check if the queue is empty.
2. If not empty, remove elements from `front` and increment `front`.

Peek:

1. Return the element at `front` if queue is not empty.

Program:

```
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0
```

Dept of AIML

```
    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)  # remove from front
        else:
            raise IndexError("Queue Underflow")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            return None

# Example usage
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

print("Queue front element:", q.peek())
print("Dequeued:", q.dequeue())
print("Queue front element after dequeue:", q.peek())
```

OUTPUT:
Queue front element: 10
Dequeued: 10
Queue front element after dequeue: 20

2. Circular Queue

Definition:

A **circular queue** is a queue in which the last position is connected back to the first position to make a circle. It efficiently uses memory by reusing empty spaces created by dequeued elements.

Advantages over linear queue:

- No “false overflow” when the queue has free space at the front after dequeuing.

Operations:

- enqueue() – add element at rear
- dequeue() – remove element from front

Dept of AIML

- isFull() – check if (rear + 1) % size == front
- isEmpty() – check if front == -1

Algorithm for Circular Queue

Enqueue:

1. Check if (rear + 1) % size == front. If yes → Queue full.
2. Else, if queue empty, set front = rear = 0.
3. Else, set rear = (rear + 1) % size.
4. Insert element at queue[rear].

Dequeue:

1. If queue empty (front == -1) → Queue underflow.
2. Retrieve element at queue[front].
3. If front == rear → only one element, reset front = rear = -1.
4. Else, front = (front + 1) % size.

Program:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def is_empty(self):
        return self.front == -1

    def is_full(self):
        return (self.rear + 1) % self.size == self.front

    def enqueue(self, item):
        if self.is_full():
            print("Queue Overflow")
            return
        if self.is_empty():
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = item

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow")
            return None
        item = self.queue[self.front]
```

Dept of AIML

```
        if self.front == self.rear:
            self.front = self.rear = -1  # queue becomes empty
        else:
            self.front = (self.front + 1) % self.size
        return item

    def peek(self):
        if self.is_empty():
            return None
        return self.queue[self.front]

    def display(self):
        if self.is_empty():
            print("Queue is empty")
            return
        i = self.front
        print("Queue elements:", end=" ")
        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()

# Example usage
cq = CircularQueue(5)
cq.enqueue(10)
cq.enqueue(20)
cq.enqueue(30)
cq.enqueue(40)
cq.display()

print("Dequeued:", cq.dequeue())
cq.display()

cq.enqueue(50)
cq.enqueue(60)  # will fill the queue
cq.display()

cq.enqueue(70)  # should show overflow
```

OUTPUT:

Queue elements: 10 20 30 40
Dequeued: 10
Queue elements: 20 30 40
Queue elements: 20 30 40 50 60

Dept of AIML